

AmnestyDAO – Milestone 2 Technical Report

Scope: Smart-contract simulations that verify functionality, security, and scalability characteristics for the DAO governance system and the Human Rights Token reward flow.

Repo: github.com/Sugarlicks/AmnestyDAO-Token/tree/add-contracts

Contracts covered:

- `soulbound_governance.ak` (minting policy for governance/membership SBT in a vault)
 - `oracle_rewards.ak` (spend validator for oracle-authorized token rewards)
-

1) Overview & Objectives

Milestone 2 requires:

1. Core governance and human-rights token contracts designed.
2. Coded & tested with clear evidence (accuracy, functionality, security, scalability).
3. Token utility, distribution, reward logic defined.
4. Technical review/audit + documentation for long-term sustainability.

This report maps each requirement to concrete tests and evidence from our simulation suite (Aiken tests with traces and execution-unit metrics).

2) Contract Summaries

2.1 `soulbound_governance.ak` — Governance SBT Mint Policy

Goal: Mint a **soulbound governance token** directly into a controlled **vault script** with an inline datum recording ownership.

Key checks implemented:

- **Treasury authorization:** minting must be signed by the `params.treasury` key.
- **Correct destination:** an output **locked by** `params.vault_hash` **must exist**.
- **Correct ownership datum:** the vault's inline datum `VaultDatum{ owner }` **must equal** the mint **recipient**.
- **No burn path:** `Burn` branch returns `False` (disallowed).

2.2 `oracle_rewards.ak` — Human Rights Token Rewards

Goal: Allow reward claims **only** when authorized by the oracle.

Key checks implemented:

- **Oracle authorization:** transaction must include the oracle's verification key hash (`extra_signatories` contains `pkh_from_vk(oracle_verification_key())`).
 - **Correct recipient & amount:** find recipient output and assert `quantity_of(value, reward_policy_id, reward_token_name) == r.amount`.
 - **No leakage:** aside from the recipient and the script's own continuity output, **no other outputs** may siphon funds.
-

3) Test Suite & Evidence

3.1 Governance / Soulbound Minting

File: `governance/soulbound_governance_test`

Tests (all PASS):

- `soulbound_mint_success`
- `soulbound_mint_fail_not_signed_by_treasury`
- `soulbound_mint_fail_not_to_vault`

- `soulbound_mint_fail_not_recipient_owner`

What they prove (by requirement):

- **Functionality:** success path mints exactly as designed into the vault with correct datum.
- **Security:** failure paths prove enforcement of (a) treasury signature, (b) correct vault script, (c) correct owner in datum.
- **Accuracy:** ownership recorded in `VaultDatum` always equals intended recipient.
- **Scalability (proxy):** we capture execution-unit ranges (see §4).

3.2 Oracle-Authorized Rewards

File: `rewards/oracle_rewards_test`

Tests (all PASS):

- `oracle_rewards_test_success`
- `oracle_rewards_test_fail_wrong_signatory`
- `oracle_rewards_test_fail_no_signatory`
- `oracle_rewards_test_fail_wrong_amount`
- `oracle_rewards_test_fail_leaked`
- `oracle_rewards_test_fail_wrong_policy_id`

What they prove (by requirement):

- **Functionality:** happy path pays correct token amount to the intended recipient.
- **Security:**
 - Only oracle-signed transactions can spend (signatory checks).
 - Amount is exact (no over/under-pay).

- No leakage to unexpected outputs.
- Using a **wrong policy id** yields 0 measured **quantity_of** and fails (guards counterfeit rewards).
- **Accuracy:** **quantity_of** check ensures the reward quantity exactly matches the redeemer.
- **Scalability (proxy):** we capture execution-unit ranges (see §4).

Run summary (screenshot evidence):

- **10 checks total: 10 passed, 0 errors, 0 warnings**
- Tests are executed with **traces and ex-unit reporting** (mem/cpu).

4) Execution-Unit (mem/cpu) Metrics & Scalability Interpretation

During **aiken check**, the framework reports **Plutus execution units** per test:

- **Governance policy (observed from screenshot):**
 - Memory (words): **~43.6K – 74.9K**
 - CPU (steps): **~14.8M – 22.9M**
- **Oracle rewards (observed):**
 - Memory (words): **~93.9K – 134.1K**
 - CPU (steps): **~32.7M – 51.1M**

How to use this for scalability evidence:

- These **per-validation costs** are a **direct proxy** for on-chain scalability because block/transaction limits are enforced in the same units (exUnits).
- [Execution-unit analysis](#) based on these figures demonstrates high scalability. Under current protocol budgets (80 M mem / 80 B CPU per block), the Soulbound

Governance policy could fit $\approx 1\,000$ txs per block, and the Oracle Rewards validator ≈ 600 txs per block before saturating resources. Both are far below protocol ceilings, confirming readiness for large-scale participation.

5) Accuracy & Functional Correctness

Mapping to requirements:

Requirement	Mechanism	Evidence
Accuracy: outputs match model	<code>VaultDatum.owner == recipient;</code> <code>quantity_of(...) == r.amount</code>	<code>soulbound_mint_success;</code> <code>oracle_rewards_test_success;</code> <code>fail_wrong_amount</code>
Functionality: core paths execute	Happy-path tests for mint and rewards	All “success” tests pass
No unintended consequences	Negative tests for signature, output routing, ownership, leakage	All “fail_*” tests pass

6) Security Properties

Threat	Defense in Contract	Test Evidence
Unauthorized minting	<code>signed_by_treasury</code> check in policy	<code>soulbound_mint_fail_not_signed_by_treasury</code>
Misrouted SBT	Enforced vault output and owner datum	<code>soulbound_mint_fail_not_to_vault,</code> <code>soulbound_mint_fail_not_recipient_owner</code>
Unauthorized rewards	Oracle signature required (<code>extra_signatories</code> contains oracle key)	<code>fail_no_signatory, fail_wrong_signatory</code>

Overpay/underpay	Exact amount check via	<code>fail_wrong_amount</code> <code>quantity_of</code>
Token leakage	Non-recipient, non-script outputs disallowed	<code>fail_leaked</code>
Counterfeit policy	Quantity counted under expected policy id only	<code>fail_wrong_policy_id</code>

7) Token Utility, Distribution & Rewards (Milestone Documentation)

Utility (summarized):

- **Governance SBT:** Non-transferable identity marker enabling governance participation, anchored in a vault with an owner datum (soulbound behavior).
- **Human Rights Token (HRT):** Reward unit issued upon oracle-verified actions (e.g., participation, contributions, verified activity), used for recognition, access gating, or programmatic incentives.

Distribution Mechanism:

- **SBT Minting:** Treasury-authorized issuance to a vault, 1:1 with identity/role enrollment.
- **HRT Rewards:** Oracle-signed attestations trigger reward transfers of exact amounts to recipients with **no leakage**, audited by the validator.

Long-term sustainability (design hooks):

- Parameterized policy (treasury PKH, vault hash) supports governance rotation.
- Oracle key can be rotated via off-chain process + parameter updates.
- Reward amounts are explicit in redeemers, enabling future programmatic budgets and caps at the application layer.
- All paths minimize exUnits to remain cost-predictable under growth.

8) Results Summary

Category	What we measured	Outcome
Functional correctness	Happy-path mint & reward flows	✓ All pass
Negative/security cases	Signature, routing, ownership, amount, leakage, policy id	✓ All pass
Scalability proxy	Per-test mem/cpu exUnits	✓ Within practical envelopes
